JobRadar - Local-First AI Job Application Assistant

Build a production-quality local-first application called **JobRadar**.

Goal

JobRadar helps users:

1. Build a professional profile from an Obsidian vault and supporting documents.
2. Analyze job descriptions and company information.
3. Generate ATS-optimized resumes.
4. Generate human-friendly resumes.
5. Tailor resumes for specific job applications.
6. Identify gaps between the candidate and the role.
7. Recommend improvements and learning paths.
8. Conduct AI-powered mock interviews.

The application will run entirely on the user's machine.

Technical Requirements

Frontend

- Next.js 15
- TypeScript
- Tailwind CSS
- shadcn/ui
- React Query
- Responsive layout

Backend

- FastAPI
- Python 3.12+
- Pydantic
- SQLite

LLM Layer

Use OpenAI-compatible APIs.

Supported providers:

- OpenAI
- OpenRouter
- Ollama
- LM Studio
- Azure OpenAI
- Any OpenAI-compatible endpoint

Configuration should be loaded from:

```
MODEL_NAME=
OPENAI_API_KEY=
OPENAI_BASE_URL=
```

Also provide a Settings page where these values can be edited.

Core User Flow

Step 1: User Profile

Create a page called:

User Profile

The user can provide data using:

Option A: Obsidian Vault

Allow selecting a local vault folder.

The backend should recursively scan:

- *.md
- *.txt

Extract:

- Experience
- Projects
- Skills
- Publications
- Certifications
- Resume versions

- Career notes

Build a unified Professional Profile.

Option B: Individual Files

Allow uploading:

- PDF
- DOCX
- Markdown
- TXT

Examples:

- Resume
- LinkedIn exports
- Skill inventory
- Publications
- Certifications

Display a summary:

- Years of experience
- Skills
- Projects
- Publications
- Certifications

Provide:

"Build Master Profile"

button.

Step 2: Role Analysis

Create a page:

Role Analysis

Accept:

- Job Description
- Job Posting
- Company Information

- Product Documents
- Research Papers
- Publications
- Markdown
- TXT
- PDF
- DOCX

Extract:

- Required skills
- Preferred skills
- Responsibilities
- Technologies
- Domain knowledge

Generate:

Role Summary

Display:

- Required skills
- Nice-to-have skills
- Seniority level
- Domain focus

Step 3: Resume Studio

Create a page:

Resume Studio

Generic Resume

Generate:

1. ATS Resume
2. Human-Friendly Resume

based on the Master Profile.

Provide preview.

Export formats:

- PDF
- DOCX
- Markdown

Tailored Resume

Generate a role-specific resume using:

- Professional Profile
- Role Analysis

Display:

- Added keywords
- Rewritten sections
- Match improvements

Export:

- PDF
- DOCX
- Markdown

Step 4: Gap Analysis

Create a page:

Gap Analysis

Compare:

Professional Profile vs Target Role

Generate:

Overall Match Score

0-100

Breakdown

- Skills Match
- Experience Match
- Domain Match

- Leadership Match

Missing Skills

List missing skills.

Evidence Mapping

Show:

Requirement -> Supporting Evidence

Example:

Requirement: Production RAG Systems

Evidence: Project XYZ

Confidence: High

Step 5: Improvement Roadmap

Generate recommendations.

Categories:

Quick Wins

Resume improvements.

Skills to Learn

Include:

- Skill
- Estimated effort
- Reason

Portfolio Projects

Suggest projects that would improve the match score.

Estimate impact.

Step 6: Resume Editor

Create an interactive editor.

For each section:

- Experience
- Skills
- Projects
- Publications

Allow:

- Accept suggestion
- Reject suggestion
- Rewrite

Show AI recommendations.

Support exporting updated resumes.

Step 7: Interview Prep

Create a page:

Interview Prep

Modes:

- Technical
- Behavioral
- Hiring Manager
- System Design
- Mixed

Difficulty:

- Easy
- Medium
- Hard

Workflow:

AI asks a question.

User answers.

AI evaluates:

- Technical depth
- Clarity
- Communication
- Structure

Provide:

- Score
- Strengths
- Weaknesses
- Improved answer

Maintain interview session history.

UI Requirements

Create a left navigation sidebar.

Sections:

- Dashboard
- User Profile
- Role Analysis
- Resume Studio
- Gap Analysis
- Improvement Plan
- Interview Prep
- Settings

Use modern shadcn/ui components.

Provide:

- Drag-and-drop uploads
- Progress indicators
- Loading states
- Error handling
- Dark mode

Architecture

Organize code into:

frontend/ backend/

Backend modules:

- document_ingestion
- obsidian_parser
- profile_builder
- role_analyzer
- resume_generator
- gap_analyzer
- interview_coach
- llm_provider

Frontend pages:

- dashboard
- user-profile
- role-analysis
- resume-studio
- gap-analysis
- improvement-plan
- interview-prep
- settings

MVP Implementation Priority

Implement in this order:

1. User Profile
2. Role Analysis
3. Generic Resume Generation
4. Tailored Resume Generation
5. Gap Analysis
6. Interview Prep

Provide working code, folder structure, API contracts, database schema, and setup instructions.

Focus on clean architecture, maintainability, and local-first operation.